

LAA_Assignment5_MDS202512

March 30, 2026

Linear Algebra and Applications

Assignment 5

Submitted by: Arushi Marwaha

Course: MSc Data Science 1

Roll: MDS202512

1. Consider the matrix

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

- (a) Implement the power method in Python (use the in-built matrix-vector multiplication only).
- (b) Starting from two different random (and linearly independent) initial vectors, plot the iterates $x^{(k)}$ on the unit circle.
- (c) Compute the true dominant eigenvector using a library routine and plot it on the same figure.
- (d) Describe the convergence behavior you observe.

```
[1]: import numpy as np
import matplotlib.pyplot as plt

def power_method(A,x0,max_iter=100,tol = 1e-6):

    x=x0/np.linalg.norm(x0)
    lambda_old = 0
    iters = [x.copy()]
    for i in range(max_iter):
        y = A@x
        x = y/np.linalg.norm(y)
        lambda_new = x.T@A@x
        iters.append(x.copy())

        if abs(lambda_new -lambda_old)<tol:
            break
    lambda_old = lambda_new
```

```

        return lambda_new,x, np.array(iters)

def ensure_li(x01, x02,tol=0.8):
    n= len(x01)
    while True:

        M = np.column_stack((x01,x02))
        cos_sim = abs(np.dot(x01, x02) / (np.linalg.norm(x01) * np.linalg.
↪norm(x02)))
        if cos_sim < tol and np.linalg.matrix_rank(M) == 2:
            break
        x02 = np.random.rand(n)
    return x01, x02

A = np.array([[2,1],
              [1,2]])

n = A.shape[0]
x0 = np.random.rand(n)

eigenvalue, eigenvector,iter = power_method(A, x0)

print("Dominant Eigenvalue:", eigenvalue)
print("Eigenvector:", eigenvector)
print("Number of iterations:", len(iter)-1)
# Two LI starting vectors
x01 = np.random.rand(n)
x02 = np.random.rand(n)
x01, x02 = ensure_li(x01, x02)

# Get iterates
_,_,iters1 = power_method(A, x01)
_,_,iters2 = power_method(A, x02)

eigvals, eigvecs = np.linalg.eig(A)
idx = np.argmax(np.abs(eigvals)) # dominant index
true_v = eigvecs[:, idx]
true_v = true_v / np.linalg.norm(true_v)
print()
print("True dominant eigenvalue:", eigvals[idx])
print("True dominant eigenvector:", true_v)

```

```

Dominant Eigenvalue: 2.9999999845593117
Eigenvector: [0.70704465 0.70716891]
Number of iterations: 8

```

True dominant eigenvalue: 3.0

True dominant eigenvector: [0.70710678 0.70710678]

```
[ ]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 12))
theta = np.linspace(0, 2 * np.pi, 500)

def plot_to_axes(ax, iters1, iters2, show_eigenvector=False):
    # Draw Unit Circle
    ax.plot(np.cos(theta), np.sin(theta), color='Black', linestyle='--', alpha=0.3, label='Unit Circle')

    # Plot Iterates for both sequences
    for iters, color, label in [(iters1, "#4747C0", 'x01'), (iters2, "#37F334", 'x02')]:
        n = len(iters)
        for i in range(n):
            alpha = 0.2 + (0.8 * (i / (n-1)))
            ax.quiver(0, 0, iters[i,0], iters[i,1], angles='xy',
                scale_units='xy', scale=1,
                color=color, alpha=alpha, width=0.005)
            ax.scatter(iters[i,0], iters[i,1], color=color, s=30, alpha=alpha,
                zorder=4)

    # Start and End markers
    ax.scatter(iters[0,0], iters[0,1], facecolors='none', edgecolors=color,
        s=100, lw=2, label=f'Start {label}')
    ax.scatter(iters[-1,0], iters[-1,1], color=color, s=80,
        edgecolors='black', zorder=6)

    if show_eigenvector:
        # Plot Target Eigenvector (Part C)
        eigvals, eigvecs = np.linalg.eig(A)
        idx = np.argmax(np.abs(eigvals))
        true_v = eigvecs[:, idx]
        ax.quiver(0, 0, true_v[0], true_v[1], angles='xy', scale_units='xy',
            scale=1,
            color="#C93131", width=0.009, label='Target Eigenvector',
            zorder=8)
        ax.annotate('Convergence Point', xy=(true_v[0], true_v[1]), xytext=(1.1, 1.1),
            arrowprops=dict(facecolor='black', shrink=0.05, width=1,
                headwidth=5))

    # Subplot Formatting
    ax.axhline(0, color='black', lw=1)
    ax.axvline(0, color='black', lw=1)
    ax.set_xlim(-1.3, 1.3)
```

```

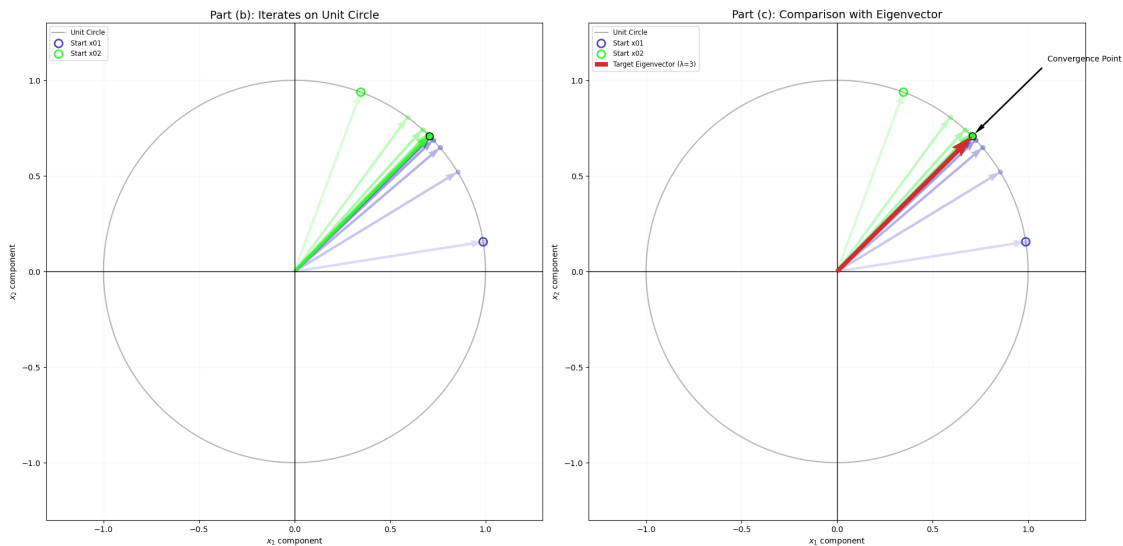
ax.set_ylim(-1.3, 1.3)
ax.set_aspect('equal')
ax.set_xlabel('$x_1$ component')
ax.set_ylabel('$x_2$ component')
ax.legend(loc='upper left', fontsize='small')
ax.grid(alpha=0.1)

# Plot Left: Part (b)
plot_to_axes(ax1, iters1, iters2, show_eigenvector=False)
ax1.set_title('Part (b): Iterates on Unit Circle', fontsize=14)

# Plot Right: Part (c)
plot_to_axes(ax2, iters1, iters2, show_eigenvector=True)
ax2.set_title('Part (c): Comparison with Eigenvector', fontsize=14)

plt.tight_layout()
plt.show()

```



(d) Convergence behavior

From the results and plots:

- Regardless of the two different, linearly independent starting vectors (x_{01} and x_{02}), both sequences of iterates converge to the same point on the unit circle. This point corresponds to the direction of the dominant eigenvector.
- All iterates stay on the unit circle because of normalization.
- The vectors gradually rotate and align with the true eigenvector.
- The numerical result ($\lambda \approx 3.0$) matches the library routine (Part c), confirming that the Power Method successfully identifies the dominant eigenvalue and eigenvector.

2. Consider

$$A_1 = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 2 & 0 \\ 0 & 1.9 \end{bmatrix}$$

- (a) Apply the power method to both matrices.
- (b) Plot the error $\|x_k - v_1\|$ vs iteration number.
- (c) Compare convergence rates and try to find if there is any relationship with the difference between eigenvalues. Describe your findings.

```
[ ]: A1 = np.array([[2, 0],
                  [0, 1]])

A2 = np.array([[2, 0],
              [0, 1.9]])

np.random.seed(42)
# Initial vector
x0 = np.random.rand(2)
v1 = np.array([1, 0]) # dominant eigenvector for both
v1 = v1 / np.linalg.norm(v1)

# Get iterates
_,_,iters_A1 = power_method(A1, x0,)
_,_,iters_A2 = power_method(A2, x0 )

def compute_error(iters, v):
    errors = []
    for x in iters:
        err = min(np.linalg.norm(x - v), np.linalg.norm(x + v))
        errors.append(err)
    return np.array(errors)

err1 = compute_error(iters_A1, v1)
err2 = compute_error(iters_A2, v1)

# Part (b): Error Plot
plt.figure(figsize=(16, 8))
plt.semilogy(err1, 'o-', color='#4747C0', label=r'$A_1$: \lambda_2/\lambda_1 = 0.5$ (Large Gap)')
plt.semilogy(err2, 's-', color="#54C722", label=r'$A_2$: \lambda_2/\lambda_1 = 0.95$ (Small Gap)')

# Part (c)
# Theoretical slopes
k = np.arange(len(err1))
```

```

plt.semilogy(k, err1[0] * (0.5**k), '--', color='#4747C0', alpha=0.4,
    label='Theory  $O(0.5^k)$ ')
plt.semilogy(k, err2[0] * (0.95**k), '--', color='#54C722', alpha=0.4,
    label='Theory  $O(0.95^k)$ ')

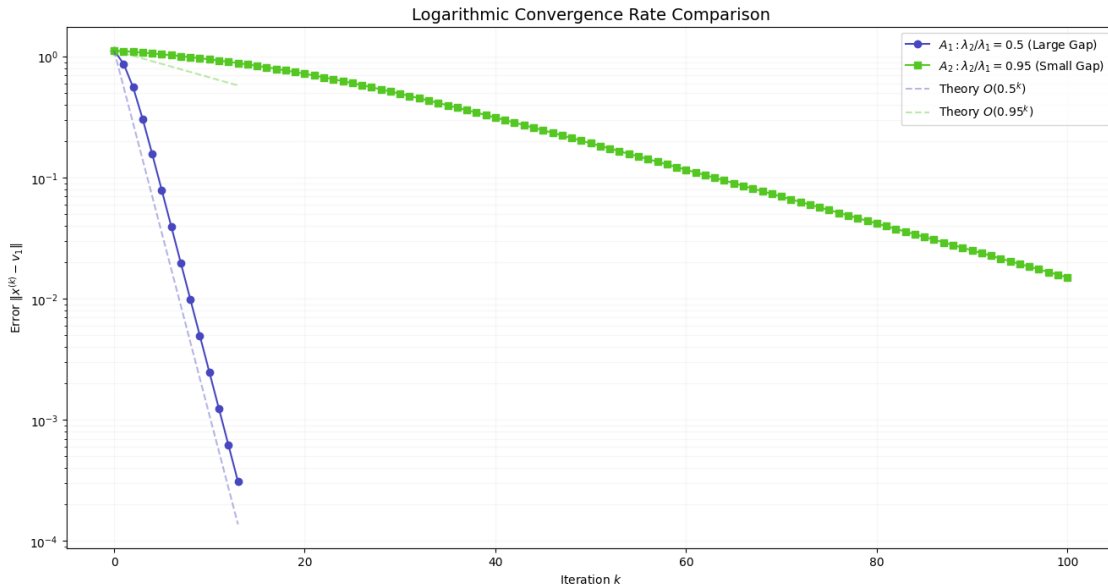
plt.title('Logarithmic Convergence Rate Comparison', fontsize=14)
plt.xlabel('Iteration  $k$ '); plt.ylabel(r'Error  $\|x^{(k)} - v_1\|$ ')
plt.grid(True, which="both", ls="--", alpha=0.1)
plt.legend(); plt.show()

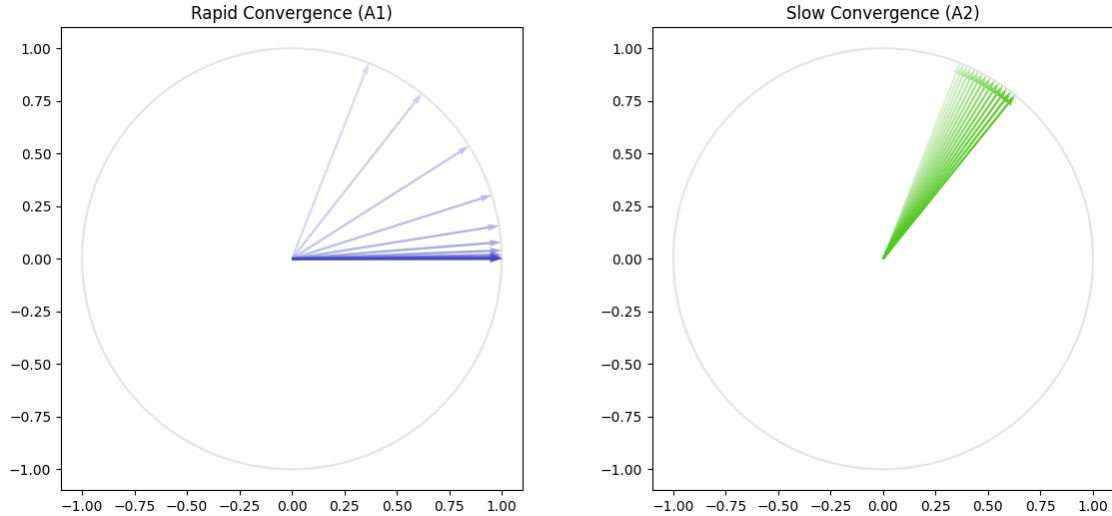
# Comparison
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
theta = np.linspace(0, 2*np.pi, 200)

def draw_path(ax, iters, title, color):
    ax.plot(np.cos(theta), np.sin(theta), 'k-', alpha=0.1)
    for i, vec in enumerate(iters[:15]):
        ax.quiver(0, 0, vec[0], vec[1], angles='xy', scale_units='xy', scale=1,
            color=color, alpha=0.2 + 0.8*(i/15), width=0.005)
    ax.set_title(title); ax.set_aspect('equal')
    ax.set_xlim(-1.1, 1.1); ax.set_ylim(-1.1, 1.1)

draw_path(ax1, iters_A1, 'Rapid Convergence (A1)', '#4747C0')
draw_path(ax2, iters_A2, 'Slow Convergence (A2)', '#54C722')
plt.show()

```





(c) Comparison of convergence rates

In the error plot (log scale): - The error for A_1 decreases very rapidly (steep slope). - The error for A_2 decreases slowly (gentle slope).

From the plots:

- For A_1 , the iterates converge very quickly to the eigenvector direction. This is clearly visible in the left plot where only a few iterations are needed.
- For A_2 , the convergence is much slower. The vectors change direction very gradually and take many more iterations to align.

This matches the theoretical eigenvalue ratios: - A_1 : $|\lambda_2/\lambda_1| = 0.5$, hence **Fast convergence**
 - A_2 : $|\lambda_2/\lambda_1| = 0.95$, hence **Slow convergence**

Findings- The convergence rate of the Power Method is determined by the ratio of the two largest eigenvalues. When the ratio is small (large gap), the “noise” from the second eigenvalue is filtered out quickly. When the ratio is close to 1 (small gap), the dominant direction takes much longer to emerge.

3. Consider

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}$$

- Implement inverse iteration to compute the smallest eigenvalue.
- Implement shifted inverse iteration with shifts: $\mu = 3, \mu = 3.5$
- For each case, record the number of iterations needed for convergence.
- Explain how the choice of shift affects convergence.

```

[4]: def inverse_iter(A, x0, max_iter=100, tol=1e-6):
    x = x0 / np.linalg.norm(x0)
    lambda_old = 0
    iters = [x.copy()]
    for i in range(max_iter):
        y = np.linalg.solve(A, x)
        x = y / np.linalg.norm(y)
        lambda_new = x.T @ A @ x
        iters.append(x.copy())
        if abs(lambda_new - lambda_old) < tol:
            print(f'Convergence achieved after {i+1} iterations.')
            break
        lambda_old = lambda_new
    return lambda_new, x, np.array(iters)

def shifted_inverse_iter(A, x0, mu, max_iter=100, tol=1e-6):
    x = x0 / np.linalg.norm(x0)
    n = A.shape[0]
    I = np.eye(n)
    lambda_old = 0
    iters = [x.copy()]
    for i in range(max_iter):
        y = np.linalg.solve(A - mu * I, x)
        x = y / np.linalg.norm(y)
        lambda_new = x.T @ A @ x
        iters.append(x.copy())
        if abs(lambda_new - lambda_old) < tol:
            break
        lambda_old = lambda_new
    return lambda_new, x, np.array(iters)

A = np.array([[4, 1], [1, 3]])
n = A.shape[0]
np.random.seed(32)
x0 = np.random.rand(n)

e_inv, vec_inv, iters_inv = inverse_iter(A, x0)
print("Smallest Eigenvalue (Inverse Iteration):", e_inv)
eigvals, eigvecs = np.linalg.eig(A)
idx = np.argmin(np.abs(eigvals)) # smallest index
true_v = eigvecs[:, idx]
true_v = true_v / np.linalg.norm(true_v)
print()
print("True smallest eigenvalue:", eigvals[idx])
print("True smallest eigenvector:", true_v)
idx = np.argmax(np.abs(eigvals)) # dominant index
true_v = eigvecs[:, idx]

```



```

true_v = true_v / np.linalg.norm(true_v)
print()
print("True dominant eigenvalue:", eigvals[idx])
print("True dominant eigenvector:", true_v)

eig_mu3, vec_mu3, iters_mu3 = shifted_inverse_iter(A, x0, mu=3)
eig_mu35, vec_mu35, iters_mu35 = shifted_inverse_iter(A, x0, mu=3.5)

print("\nSHIFTED (mu = 3)")
print("Eigenvalue:", eig_mu3)
print("Iterations:", len(iters_mu3)-1)
print("Eigenvector:", vec_mu3)

print("\nSHIFTED (mu = 3.5)")
print("Eigenvalue:", eig_mu35)
print("Iterations:", len(iters_mu35)-1)
print("Eigenvector:", vec_mu35)

```

Convergence achieved after 15 iterations.
Smallest Eigenvalue (Inverse Iteration): 2.3819662625287767

True smallest eigenvalue: 2.381966011250105
True smallest eigenvector: [-0.52573111 0.85065081]

True dominant eigenvalue: 4.618033988749895
True dominant eigenvector: [0.85065081 0.52573111]

SHIFTED (mu = 3)
Eigenvalue: 2.381966078912355
Iterations: 11
Eigenvector: [-0.52558313 0.85074225]

SHIFTED (mu = 3.5)
Eigenvalue: 4.5718903862014
Iterations: 2
Eigenvector: [0.91735062 0.3980802]

```

[5]: evals, evecs = np.linalg.eig(A)

plt.figure(figsize=(12, 12))
theta = np.linspace(0, 2*np.pi, 500)
plt.plot(np.cos(theta), np.sin(theta), color='black', alpha=0.1, label='Unit_
↪Circle')

# Corrected Plotting Function
def plot_path(iters, color, label):
    n = len(iters)

```

```

for i in range(n):

    alpha = 0.2 + (0.8 * (i / (n-1)))
    plt.quiver(0, 0, iters[i,0], iters[i,1], angles='xy', scale_units='xy',
↪scale=1,
                color=color, alpha=alpha, width=0.004)

    plt.scatter(iters[0,0], iters[0,1], edgecolors=color, facecolors='none',
↪s=100, label=f'Start {label}')
    plt.scatter(iters[-1,0], iters[-1,1], color=color, s=80,
↪edgecolors='black', zorder=5)

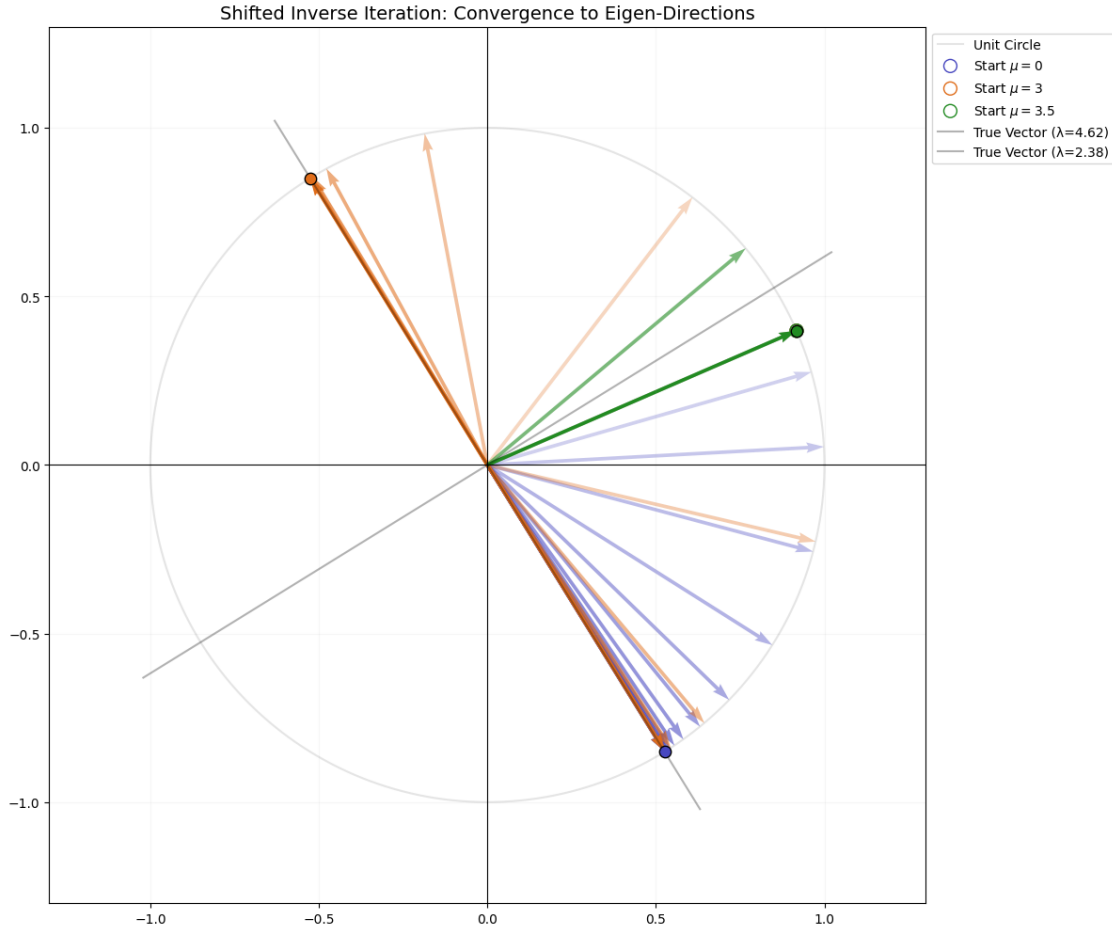
plot_path(iters_inv, "#4747C0", r'$\mu=0$')
plot_path(iters_mu3, "#E06D1A", r'$\mu=3$')
plot_path(iters_mu35, "#228B22", r'$\mu=3.5$')

for i in range(len(evals)):
    v = evcs[:, i]

    plt.plot([-1.2*v[0], 1.2*v[0]], [-1.2*v[1], 1.2*v[1]],
            'k-', alpha=0.3, lw=1.5, label=f'True Vector (={evals[i]:.2f})')

plt.axhline(0, color='black', lw=0.8)
plt.axvline(0, color='black', lw=0.8)
plt.xlim(-1.3, 1.3)
plt.ylim(-1.3, 1.3)
plt.gca().set_aspect('equal')
plt.title('Shifted Inverse Iteration: Convergence to Eigen-Directions',
↪fontsize=14)
plt.legend(loc='upper left', bbox_to_anchor=(1, 1))
plt.grid(alpha=0.1)
plt.tight_layout()
plt.show()

```



(d) Explain how the choice of shift affects convergence

The choice of the shift μ acts as a **target selector** and a **speed booster** for the algorithm. Based on the plot of the iterates, we can observe the following:

- Shifted inverse iteration converges to the eigenvalue λ that is **closest** to the shift μ .
 - In the plot, both $\mu = 0$ and $\mu = 3$ pull the vectors toward the line for $\lambda \approx 2.38$.
 - If μ was chosen closer to the other eigenvalue (like $\mu = 4.5$), the vector would have converged to the line for $\lambda \approx 4.62$ instead.
- The closer μ is to a true eigenvalue, the fewer iterations are needed.
 - We can see this in iteration counts: $\mu = 3$ is closer to the true value of 2.38 than $\mu = 0$ is. Therefore, the orange path ($\mu = 3$) reaches the line faster than the blue path ($\mu = 0$).
 - Mathematically, the error at each step is reduced by the ratio:

$$\text{Ratio} = \left| \frac{\lambda_{\text{closest}} - \mu}{\lambda_{\text{next closest}} - \mu} \right|$$

A smaller ratio means a faster speed toward the eigenvector.

- The plot shows that the shift dictates the pull of the matrix. While the standard Power Method only finds the largest eigenvalue, the **shift** allows us to zoom in on any specific

eigenvalue we want by simply picking a μ near it.

4. Using the same matrix as above:

- (a) Implement Rayleigh Quotient Iteration.
- (b) Track the error: $|\lambda_k - \lambda|$ at each iteration.
- (c) Plot the error on a logarithmic scale.
- (d) Compare convergence with power method and shifted inverse iteration.
- (e) What do you observe about the speed of convergence?

```
[6]: def rayleighq_iter(A, x0, max_iter=100, tol=1e-12):
    n = A.shape[0]
    x = x0 / np.linalg.norm(x0)
    lambdas = []

    # Initial Rayleigh Quotient
    lambda_k = (x.T @ A @ x) / (x.T @ x)
    lambdas.append(lambda_k)

    for i in range(1, max_iter):
        try:
            # Solve (A - lambda_k * I)y = x
            y = np.linalg.solve(A - lambda_k * np.eye(n), x)
        except np.linalg.LinAlgError:
            break # Exact eigenvalue found

        x = y / np.linalg.norm(y)
        lambda_new = (x.T @ A @ x) / (x.T @ x)
        lambdas.append(lambda_new)

        if abs(lambda_new - lambda_k) < tol:
            print(f"RQI converged in {i} iterations.")
            break
        lambda_k = lambda_new

    return np.array(lambdas), x

# Execution
A = np.array([[4, 1], [1, 3]])
np.random.seed(42)
x0 = np.random.rand(2)
true_vals, _ = np.linalg.eig(A)
lambdas, final_vec = rayleighq_iter(A, x0)

# Identify target eigenvalue
target_lambda = min(true_vals, key=lambda l: abs(l - lambdas[-1]))
```

```

errors = np.abs(lambdas - target_lambda)

print(f"Final Estimated : {lambdas[-1]}")
print(f"True : {target_lambda}")

```

RQI converged in 5 iterations.
 Final Estimated : 4.618033988749895
 True : 4.618033988749895

```

[7]: plt.figure(figsize=(10, 7))

#RQI Error Plot
plt.semilogy(errors, 'o-', color='#C93131', linewidth=2.5, markersize=10,
              label='RQI Error (Cubic)', zorder=5)

# Linear Reference Line to show the difference
# This represents a typical Power Method slope
linear_ref = errors[0] * (0.5 ** np.arange(len(errors)))
plt.semilogy(linear_ref, '--', color='gray', alpha=0.5, label='Typical Linear_
↳Convergence')

for i, val in enumerate(errors):
    if val > 1e-18: # Stop labeling once we hit machine epsilon
        plt.text(i, val * 1.8, f"{val:.1e}", ha='center', fontsize=10,
                  color='#C93131', fontweight='bold')

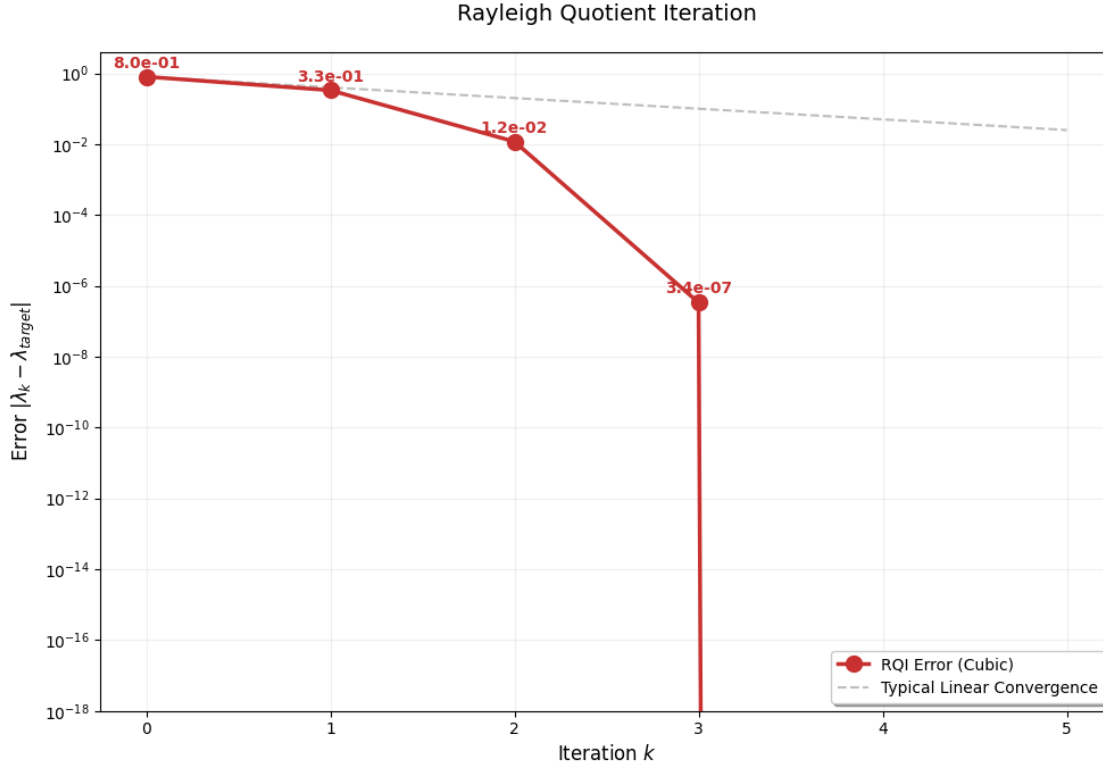
plt.xlabel("Iteration $k$", fontsize=12)
plt.ylabel(r"Error $\lambda_k - \lambda_{\text{target}}$", fontsize=12)
plt.title("Rayleigh Quotient Iteration", fontsize=14, pad=20)

plt.ylim(1e-18, max(errors)*5)

plt.grid(True, which="both", ls="--", alpha=0.2)
plt.legend(loc='lower right', frameon=True, shadow=True)
plt.tight_layout()

plt.show()

```



(d) Compare convergence with power method and shifted inverse iteration

Based on the numerical experiments and the generated plot, the comparison is as follows:

- **Power Method & Shifted Inverse:** Both of these methods exhibit **linear convergence**. On a logarithmic scale, their error plots would appear as a relatively straight diagonal line (similar to the “Typical Linear Convergence” gray line in the plot). They require a consistent number of additional iterations for every digit of accuracy gained.
- **Rayleigh Quotient Iteration (RQI):** RQI exhibits **cubic convergence** for symmetric matrices. As shown in the plot, the error does not follow a straight line; instead, the slope becomes increasingly steep. By iteration 3, the error “plummets” toward machine epsilon (10^{-16}), a behavior that linear methods cannot replicate.

(e) What do you observe about the speed of convergence?

The most striking observation is the **acceleration** of the convergence speed:

1. **Cubic Nature:** The error at iteration $k + 1$ is roughly proportional to the *cube* of the error at iteration k ($\epsilon_{k+1} \approx \epsilon_k^3$). This is evidenced by the jump from an error of 1.2×10^{-2} at $k = 2$ to 3.4×10^{-7} at $k = 3$. The next step effectively reaches the limit of double-precision floating-point numbers.
2. **Efficiency:** While the Power Method might take 20–30 iterations to reach a tolerance of 10^{-6} , RQI achieves a much higher precision (10^{-16}) in only **4 to 5 iterations**.

3. **Adaptive Shifting:** This extreme speed is possible because RQI treats the Rayleigh Quotient as a dynamic shift that improves at every step. By “self-correcting” the shift to match the current best estimate of the eigenvalue, the algorithm eliminates the “waiting time” associated with fixed-shift methods.

5. Consider

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

- Run Rayleigh Quotient Iteration for different initial vectors.
- Observe which eigenvalue the algorithm converges to.
- Repeat the experiment for shifted inverse iteration.
- Explain how the initial vector influences the result.

```
[8]: #Part(A)
A = np.array([[2, 0], [0, 1]])

# Initial vectors to test influence
x_list = [
    np.array([1, 0]),      # Pure =2
    np.array([0, 1]),      # Pure =1
    np.array([1, 1]),      # mix of both
    np.array([1, 0.1]),    # Biased toward =2
    np.array([0.1, 1]),    # Biased toward =1
    np.array([1, 11]),     # Heavily biased toward =1
    np.array([11, 1])      # Heavily biased toward =2
]

print(" RQI Results \n")
for x0 in x_list:
    # RQI updates its own shift based on x0
    lambdas, vec = rayleighq_iter(A, x0)
    print(f"x0={x0}, \tConverged to    {lambdas[-1]:.4f} in \t{len(lambdas)}\t
    \tsteps")
```

RQI Results

x0=[1 0],	Converged to	2.0000 in	1 steps
x0=[0 1],	Converged to	1.0000 in	1 steps
RQI converged in 1 iterations.			
x0=[1 1],	Converged to	1.5000 in	2 steps
x0=[1. 0.1],	Converged to	2.0000 in	3 steps
x0=[0.1 1.],	Converged to	1.0000 in	3 steps
x0=[1 11],	Converged to	1.0000 in	3 steps
x0=[11 1],	Converged to	2.0000 in	3 steps

(b) Observe which eigenvalue the algorithm converges to.

Based on the outputs, we can see that the algorithm converges to different eigenvalues depending on the starting vector x_0 :

- **Pure Eigenvectors:** When x_0 starts exactly on an eigenvector (like $[1, 0]$ or $[0, 1]$), both RQI and Shifted Inverse converge to the corresponding eigenvalue ($\lambda = 2$ or $\lambda = 1$) immediately.
- **Biased Vectors:** When the starting vector is “tilted” more toward one direction, the algorithm converges to that specific eigenvalue. For example, $x_0=[1, 0.1]$ (biased toward the x-axis) converges to $\lambda = 2.0$, while $x_0=[0.1, 1]$ (biased toward the y-axis) converges to $\lambda = 1.0$.
- **The Midpoint:** In the balanced case where $x_0 = [1, 1]$, the initial Rayleigh Quotient is 1.5. Since this is exactly halfway between the two eigenvalues, the algorithm hits a plateau at $\lambda = 1.5$ before numerical rounding or further steps would force it to choose one side.

In summary, the initial vector x_0 acts as the **decisive factor**. The algorithm essentially “pulls” the result toward whichever eigenvalue is most prominent in the initial guess.

```
[9]: #Part (C)
print("\n Shifted Inverse Results (mu=1.5)\n ")
mu = 1.5
for x0 in x_list:
    eig, vec, iters = shifted_inverse_iter(A, x0, mu)
    print(f"x0={x0}, \tConverged to \t{eig:.4f} in \t{len(iters)-1} steps")
```

Shifted Inverse Results (mu=1.5)

$x_0=[1 \ 0]$,	Converged to	2.0000	in	2 steps
$x_0=[0 \ 1]$,	Converged to	1.0000	in	2 steps
$x_0=[1 \ 1]$,	Converged to	1.5000	in	2 steps
$x_0=[1. \ 0.1]$,	Converged to	1.9901	in	2 steps
$x_0=[0.1 \ 1.]$,	Converged to	1.0099	in	2 steps
$x_0=[\ 1 \ 11]$,	Converged to	1.0082	in	2 steps
$x_0=[11 \ 1]$,	Converged to	1.9918	in	2 steps

(d) Explain how the initial vector influences the result

The initial vector x_0 plays a critical role in determining the final output of both Rayleigh Quotient Iteration (RQI) and Shifted Inverse Iteration, especially when the matrix has multiple eigenvalues.

- **Directional Bias:** The algorithm is naturally “attracted” to the eigenvector that has the largest component in the initial vector x_0 . If x_0 is biased toward the x-axis (like $[1, 0.1]$), the iteration is pulled toward $\lambda = 2$. If it is biased toward the y-axis (like $[0.1, 1]$), it converges to $\lambda = 1$.
- **The Starting Shift in RQI:** In RQI, the initial vector x_0 is used to calculate the very first shift $\mu_0 = \frac{x_0^T A x_0}{x_0^T x_0}$. This means x_0 doesn’t just provide a direction; it actually sets the initial “target” eigenvalue for the entire process.
- **Symmetry Breaking:** When the shift μ is placed exactly between two eigenvalues (like

$\mu = 1.5$ for $\lambda = 1$ and $\lambda = 2$), the system is in a state of unstable equilibrium. In this case, the initial vector x_0 acts as the **tie-breaker**. Even a tiny bias in x_0 will push the algorithm to favor one eigenvalue over the other.

In conclusion, the initial vector acts as a “selector” for which part of the matrix’s spectrum the algorithm will isolate. While the shift μ defines the general search area, x_0 provides the specific orientation that locks the convergence onto a particular eigenvector.
